

```
# from google.colab import files
# import zipfile, os

# # 💎 1. Upload the ZIP file from your local computer
# uploaded = files.upload()      # choose HINDI_DATASET-20251027T0918

# # 💎 2. Extract it
# zip_path = "HINDI_DATASET-20251027T091805Z-1-001.zip"
# extract_path = "hindi_dataset"

# with zipfile.ZipFile(zip_path, 'r') as zip_ref:
#     zip_ref.extractall(extract_path)

# print("✅ Files extracted to:", extract_path)
# print(os.listdir(extract_path)[:10])      # show first few files
```

```
from google.colab import drive
drive.mount('/content/drive')
```

Mounted at /content/drive

```
!google-drive-ocamlfuse /content/drive
```

```
/bin/bash: line 1: google-drive-ocamlfuse: command not found
```

```
!ls /content/drive/MyDrive | head
```

```
ADT SPECIFICATION OF QUEUE.gdoc
ADT SPECIFICATION OF STACK.gdoc
archive (3).zip
archive.zip
Augmented Reality in Education: Understanding Its Impact .gform
Augmented Reality in Education: Understanding Its Impact (Responses
Book1.csv
Canva.gform
Certify'em Records.gsheets
chi_square_test_and_its_application_in_hypothesis.17.pdf
```

```
!ls /content/drive/MyDrive/HINDI_DATASET | head
```

```
hin_tourism_set01.txt
hin_tourism_set02.txt
hin_tourism_set03.txt
hin_tourism_set04.txt
hin_tourism_set05.txt
hin_tourism_set06.txt
hin_tourism_set07.txt
hin_tourism_set08.txt
hin_tourism_set09.txt
hin_tourism_set10.txt
```

```
data_folder = "/content/drive/MyDrive/HINDI_DATASET"
```

```
import os

sentences = []

for filename in os.listdir(data_folder):
    if filename.endswith(".txt"):
        file_path = os.path.join(data_folder, filename)
        with open(file_path, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if not line:
                    continue
                parts = line.split("\t")
                if len(parts) < 2:
                    continue
                tokens = parts[1].split()
                word_tag_pairs = []
                for token in tokens:
                    if "\\" in token:
                        word, tag = token.split("\\", 1)
                        word_tag_pairs.append((word, tag))
                sentences.append(word_tag_pairs)

print(f"✅ Loaded {len(sentences)} sentences.")
print("Example:", sentences[0][:10])
```

```
✅ Loaded 25025 sentences.
Example: []
```

```

import re, unicodedata

# Separate words and tags
all_words = [[w for w, t in sent] for sent in sentences]
all_tags = [[t for w, t in sent] for sent in sentences]

# Normalization function
def clean_text(text):
    text = unicodedata.normalize('NFKC', text)
    text = re.sub(r'["'"]', '', text)
    text = re.sub(r'[\u200c\u200d]', '', text) # remove zero-width
    text = re.sub(r'\s+', ' ', text).strip()
    return text

# Apply cleaning
cleaned_sentences = [[clean_text(w) for w in s] for s in all_words]

print(f"✅ Cleaned {len(cleaned_sentences)} sentences.")
print("Example cleaned sentence:", cleaned_sentences[0])

```

✅ Cleaned 25025 sentences.
Example cleaned sentence: []

```

import numpy as np
from collections import Counter

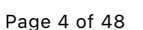
num_sentences = len(cleaned_sentences)
num_tokens = sum(len(s) for s in cleaned_sentences)
unique_words = sorted({w for s in cleaned_sentences for w in s})
unique_tags = sorted({t for ts in all_tags for t in ts})

print(f"📈 Total Sentences: {num_sentences}")
print(f"🪄 Total Tokens: {num_tokens}")
print(f"🆕 Unique Words: {len(unique_words)}")
print(f"📁 Unique POS Tags: {len(unique_tags)}")
print("POS Tags List:", unique_tags)

```

📈 Total Sentences: 25025
 🪄 Total Tokens: 424087
 🆕 Unique Words: 28579
 📁 Unique POS Tags: 112
 POS Tags List: ['', 'CCC_CD', 'CC_CCD', 'CC_CCS', 'CC_CS', 'C_CCD',

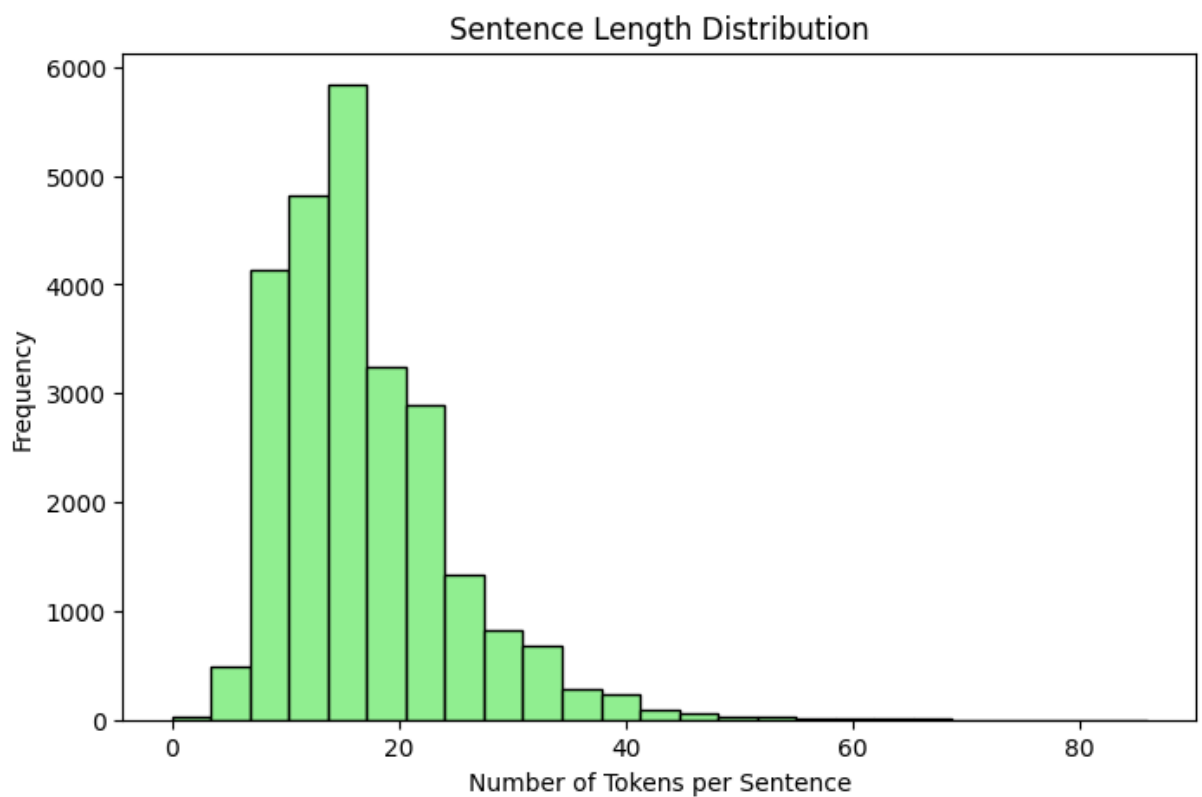
```
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:1
fig.canvas.print_figure(bytes_io, **kw)
/usr/local/lib/python3.12/dist-packages/IPython/core/pylabtools.py:1
fig.canvas.print_figure(bytes_io, **kw)
```



```
sent_lengths = [len(s) for s in cleaned_sentences]

plt.figure(figsize=(8,5))
plt.hist(sent_lengths, bins=25, color='lightgreen', edgecolor='black')
plt.title("Sentence Length Distribution")
plt.xlabel("Number of Tokens per Sentence")
plt.ylabel("Frequency")
plt.show()

print(f"Average Sentence Length: {np.mean(sent_lengths):.2f}")
print(f"Max Sentence Length: {np.max(sent_lengths)}")
print(f"Min Sentence Length: {np.min(sent_lengths)}")
```



Average Sentence Length: 16.95
Max Sentence Length: 86
Min Sentence Length: 0

```
word_counts = Counter([w for s in cleaned_sentences for w in s])
print("⬆️ Top 20 Frequent Words:")
for word, count in word_counts.most_common(20):
    print(f"{word:<20} {count}")

print("\n🟡 Top 10 Frequent POS Tags:")
for tag, count in tag_counts.most_common(10):
    print(f"{tag:<10} {count}")
```

⬆️ Top 20 Frequent Words:

	24453
के	15941
है	15718
में	12663
की	9824
,	9585
से	9193
हैं	7374
का	7272
और	4819
पर	4814
को	3840
भी	3409
एक	3096
लिए	2607
यहाँ	2494
इस	2224
ही	2048
यह	2033
"	1958

🟡 Top 10 Frequent POS Tags:

N_NN	110579
PSP	69358
N_NNP	44687
RD_PUNC	39487
V_VM	37012
V_VAUX	33085
JJ	21707
QT_QTC	10886
CC_CCD	9058
DM_DMD	7808

```
from sklearn.model_selection import train_test_split

train_sents, test_sents, train_tags, test_tags = train_test_split(
    cleaned_sentences, all_tags, test_size=0.2, random_state=42
)

val_sents, test_sents, val_tags, test_tags = train_test_split(
    test_sents, test_tags, test_size=0.5, random_state=42
)

print(f"Train: {len(train_sents)} | Validation: {len(val_sents)} |
```

```
Train: 20020 | Validation: 2502 | Test: 2503
```

Rule-Based POS Tagging

```

# model.py (inline in Colab)
import re, pickle
from collections import defaultdict, Counter

class RuleBasedPOSTagger:
    def __init__(self):
        self.word_tag_freq = defaultdict(Counter)
        self.rules = []
        self.default_tag = "UNK"

    def train(self, sentences, tags):
        for s, t in zip(sentences, tags):
            for word, tag in zip(s, t):
                self.word_tag_freq[word][tag] += 1

        self.rules = [
            (re.compile(r"(\u094d|\u094e|\u094f|\u0950)$"), "N_NN"),
            (re.compile(r"(\u0940|\u0941|\u0942|\u0943|\u0944|\u0945|\u0946|\u0947|\u0948|\u0949|\u094a|\u094b|\u094c|\u094d|\u094e|\u094f|\u0950|\u0951|\u0952|\u0953|\u0954|\u0955|\u0956|\u0957|\u0958|\u0959|\u095a|\u095b|\u095c|\u095d|\u095e|\u095f|\u0960|\u0961|\u0962|\u0963|\u0964|\u0965|\u0966|\u0967|\u0968|\u0969|\u096a|\u096b|\u096c|\u096d|\u096e|\u096f|\u0970|\u0971|\u0972|\u0973|\u0974|\u0975|\u0976|\u0977|\u0978|\u0979|\u097a|\u097b|\u097c|\u097d|\u097e|\u097f|\u0980|\u0981|\u0982|\u0983|\u0984|\u0985|\u0986|\u0987|\u0988|\u0989|\u098a|\u098b|\u098c|\u098d|\u098e|\u098f|\u0990|\u0991|\u0992|\u0993|\u0994|\u0995|\u0996|\u0997|\u0998|\u0999|\u099a|\u099b|\u099c|\u099d|\u099e|\u099f|\u09a0|\u09a1|\u09a2|\u09a3|\u09a4|\u09a5|\u09a6|\u09a7|\u09a8|\u09a9|\u09aa|\u09ab|\u09ac|\u09ad|\u09ae|\u09af|\u09b0|\u09b1|\u09b2|\u09b3|\u09b4|\u09b5|\u09b6|\u09b7|\u09b8|\u09b9|\u09ba|\u09bb|\u09bc|\u09bd|\u09be|\u09bf|\u09c0|\u09c1|\u09c2|\u09c3|\u09c4|\u09c5|\u09c6|\u09c7|\u09c8|\u09c9|\u09ca|\u09cb|\u09cc|\u09cd|\u09ce|\u09cf|\u09d0|\u09d1|\u09d2|\u09d3|\u09d4|\u09d5|\u09d6|\u09d7|\u09d8|\u09d9|\u09da|\u09db|\u09dc|\u09dd|\u09de|\u09df|\u09e0|\u09e1|\u09e2|\u09e3|\u09e4|\u09e5|\u09e6|\u09e7|\u09e8|\u09e9|\u09ea|\u09eb|\u09ec|\u09ed|\u09ee|\u09ef|\u09f0|\u09f1|\u09f2|\u09f3|\u09f4|\u09f5|\u09f6|\u09f7|\u09f8|\u09f9|\u09fa|\u09fb|\u09fc|\u09fd|\u09fe|\u09ff)$"), "V_VM"),
            (re.compile(r"(\u0940|\u0941|\u0942|\u0943|\u0944|\u0945|\u0946|\u0947|\u0948|\u0949|\u094a|\u094b|\u094c|\u094d|\u094e|\u094f|\u0950|\u0951|\u0952|\u0953|\u0954|\u0955|\u0956|\u0957|\u0958|\u0959|\u095a|\u095b|\u095c|\u095d|\u095e|\u095f|\u0960|\u0961|\u0962|\u0963|\u0964|\u0965|\u0966|\u0967|\u0968|\u0969|\u096a|\u096b|\u096c|\u096d|\u096e|\u096f|\u0970|\u0971|\u0972|\u0973|\u0974|\u0975|\u0976|\u0977|\u0978|\u0979|\u097a|\u097b|\u097c|\u097d|\u097e|\u097f|\u0980|\u0981|\u0982|\u0983|\u0984|\u0985|\u0986|\u0987|\u0988|\u0989|\u098a|\u098b|\u098c|\u098d|\u098e|\u098f|\u0990|\u0991|\u0992|\u0993|\u0994|\u0995|\u0996|\u0997|\u0998|\u0999|\u099a|\u099b|\u099c|\u099d|\u099e|\u099f|\u09a0|\u09a1|\u09a2|\u09a3|\u09a4|\u09a5|\u09a6|\u09a7|\u09a8|\u09a9|\u09aa|\u09ab|\u09ac|\u09ad|\u09ae|\u09af|\u09b0|\u09b1|\u09b2|\u09b3|\u09b4|\u09b5|\u09b6|\u09b7|\u09b8|\u09b9|\u09ba|\u09bb|\u09bc|\u09bd|\u09be|\u09bf|\u09c0|\u09c1|\u09c2|\u09c3|\u09c4|\u09c5|\u09c6|\u09c7|\u09c8|\u09c9|\u09ca|\u09cb|\u09cc|\u09cd|\u09ce|\u09cf|\u09d0|\u09d1|\u09d2|\u09d3|\u09d4|\u09d5|\u09d6|\u09d7|\u09d8|\u09d9|\u09da|\u09db|\u09dc|\u09dd|\u09de|\u09df|\u09e0|\u09e1|\u09e2|\u09e3|\u09e4|\u09e5|\u09e6|\u09e7|\u09e8|\u09e9|\u09ea|\u09eb|\u09ec|\u09ed|\u09ee|\u09ef|\u09f0|\u09f1|\u09f2|\u09f3|\u09f4|\u09f5|\u09f6|\u09f7|\u09f8|\u09f9|\u09fa|\u09fb|\u09fc|\u09fd|\u09fe|\u09ff)$"), "PSP"),
            (re.compile(r"^[0-9]+$"), "QC"),
            (re.compile(r"[!,?;:.]+$"), "RD_PUNC"),
        ]

    def predict(self, sentence):
        tags = []
        for word in sentence:
            if word in self.word_tag_freq:
                tag = self.word_tag_freq[word].most_common(1)[0][0]
            else:
                tag = self.default_tag
                for pattern, rule_tag in self.rules:
                    if pattern.search(word):
                        tag = rule_tag
                        break
            tags.append(tag)
        return tags

    def save(self, path):
        with open(path, "wb") as f:
            pickle.dump(self, f)

    @staticmethod
    def load(path):
        with open(path, "rb") as f:
            return pickle.load(f)

```



```
# Initialize and train
rule_tagger = RuleBasedPOSTagger()
rule_tagger.train(train_sents, train_tags)
rule_tagger.save("model.pkl")
print("✅ Model saved as model.pkl")

# Predict on validation set
y_true, y_pred = [], []
for s, t in zip(val_sents, val_tags):
    pred = rule_tagger.predict(s)
    y_true.extend(t)
    y_pred.extend(pred)
```

✅ Model saved as model.pkl

```
from sklearn.metrics import classification_report, accuracy_score

print("✅ Rule-Based POS Tagger Evaluation:")
print("Accuracy:", accuracy_score(y_true, y_pred))
print("\nDetailed Report:\n")
print(classification_report(y_true, y_pred, zero_division=0))
```

✅ Rule-Based POS Tagger Evaluation:
Accuracy: 0.8279681143743974

Detailed Report:

	precision	recall	f1-score	support
CC_CCD	0.94	0.95	0.94	923
CC_CCS	0.73	0.77	0.75	281
DM_DMD	0.69	0.87	0.77	784
DM_DMI	0.55	0.85	0.67	66
DM_DMQ	0.25	0.12	0.17	16
DM_DMR	0.34	0.15	0.20	234
JJ	0.75	0.71	0.73	2168
N-NN	0.00	0.00	0.00	1
NN_NNP	0.00	0.00	0.00	1
N_N	0.00	0.00	0.00	1
N_NN	0.84	0.83	0.84	11268
N_NNN	0.00	0.00	0.00	1
N_NNP	0.86	0.69	0.77	4356
N_NP	0.00	0.00	0.00	1
N_NST	0.69	0.74	0.71	510
PR_PRC	1.00	1.00	1.00	1
PR_PRF	0.97	0.95	0.96	155
PR_PRI	0.00	0.00	0.00	49
PR_PRL	0.83	0.73	0.78	329

PR_PRP	0.84	0.87	0.86	463
PR_PRQ	0.61	0.70	0.65	20
PR_PRR	0.00	0.00	0.00	1
PR_RPL	0.00	0.00	0.00	1
PR_RPP	0.00	0.00	0.00	1
PSP	0.97	0.99	0.98	6918
QC	0.00	0.00	0.00	0
QT_QT	0.00	0.00	0.00	1
QT_QTC	0.94	0.89	0.92	1044
QT_QTF	0.70	0.70	0.70	583
QT_QTO	0.84	0.75	0.79	96
RB	0.36	0.17	0.23	106
RD_ECH	0.11	0.03	0.05	31
RD_PUNC	0.99	0.98	0.98	4024
RD_RDF	1.00	0.11	0.20	9
RD_SYM	0.40	0.95	0.56	58
RP_INJ	1.00	0.33	0.50	3
RP_INTF	0.54	0.85	0.66	112
RP_NEG	0.95	1.00	0.97	129
RP_PRD	0.00	0.00	0.00	1
RP_RPD	0.95	0.88	0.91	761
UNK	0.00	0.00	0.00	0
VM	0.00	0.00	0.00	2
V_VAUX	0.74	0.86	0.80	3323
V_VM	0.74	0.61	0.67	3694
_NNP	0.00	0.00	0.00	0
n-nn	0.00	0.00	0.00	1
accuracy			0.83	42527
macro avg	0.48	0.46	0.45	42527
weighted avg	0.85	0.83	0.84	42527

```
for i in range(3):
    print("Sentence:", " ".join(val_sents[i]))
    print("True Tags:", val_tags[i])
    print("Predicted:", rule_tagger.predict(val_sents[i]))
    print("-"*80)
```

Sentence: इस कला महोत्सव का मुख्य मकसद विभिन्न देशों के कलाविदों के बीच सांस्कृतिक व व
True Tags: ['DM_DMD', 'N_NN', 'N_NN', 'PSP', 'JJ', 'N_NN', 'JJ', 'N_
Predicted: ['DM_DMD', 'N_NN', 'N_NN', 'PSP', 'JJ', 'N_NN', 'JJ', 'N_

Sentence: मकाऊ में जेटी से बाहर निकलते ही रिक्शे पर ऊँघते रिक्शा चालकों को देख कर अजीब
True Tags: ['N_NNP', 'PSP', 'N_NN', 'PSP', 'N_NST', 'V_VM', 'RP_RPD'
Predicted: ['N_NNP', 'PSP', 'N_NN', 'PSP', 'N_NST', 'V_VM', 'RP_RPD'

Sentence: इसी से मिलती - जुलती स्केटिंग है सिंक्रेनाइज्ड स्केटिंग ।
True Tags: ['PR_PRL', 'PSP', 'V_VM', 'RD_PUNC', 'RD_ECH', 'N_NN', 'V_
Predicted: ['DM_DMD', 'PSP', 'V_VM', 'RD_PUNC', 'N_NN', 'N_NN', 'V_V

```
# quick reload + test
model_test = RuleBasedPOSTagger.load("model.pkl")
sample = ["मैं", "स्कूल", "जा", "रहा", "हूँ", "।"]
print("Sentence:", sample)
print("Predicted Tags:", model_test.predict(sample))
```

Sentence: ['मैं', 'स्कूल', 'जा', 'रहा', 'हूँ', '।']
Predicted Tags: ['PR_PRP', 'N_NN', 'V_VAUX', 'V_VAUX', 'V_VAUX', 'RD

```
import gradio as gr

# Load model
tagger = RuleBasedPOSTagger.load("model.pkl")

def tokenize_hindi(text):
    return text.strip().split()

def predict_tags(text):
    tokens = tokenize_hindi(text)
    tags = tagger.predict(tokens)
    return "\n".join(f"{w} / {t}" for w, t in zip(tokens, tags))

with gr.Blocks() as demo:
    gr.Markdown("# 🇮🇳 Hindi Rule-Based POS Tagger")
    gr.Markdown("Enter a Hindi sentence below to get POS tags.")
```

```
inp = gr.Textbox(lines=4, placeholder="यहाँ वाक्य लिखें...")
out = gr.Textbox(lines=8)
btn = gr.Button("Tag Sentence")
btn.click(fn=predict_tags, inputs=inp, outputs=out)

demo.launch(share=True) # use share=True to test it directly from
```

Colab notebook detected. To show errors in colab notebook, set debug
* Running on public URL: <https://dbf9fabe19ed38e01d.gradio.live>

This share link expires in 1 week. For free permanent hosting and GP



Hindi Rule-Based POS Tagger

Enter a Hindi sentence below to get POS tags.

Textbox

यहाँ वाक्य लिखें...

Textbox

Hidden Markov Model (HMM) POS Tagger

```
from collections import defaultdict, Counter
import numpy as np
```

```

class HMPOSTagger:
    def __init__(self):
        self.transition_probs = defaultdict(lambda: defaultdict(float))
        self.emission_probs = defaultdict(lambda: defaultdict(float))
        self.tags = set()
        self.vocab = set()
        self.tag_counts = Counter()
        self.start_tag = "<s>"
        self.end_tag = "</s>"

    def train(self, sentences, tags):
        # Count tag and word occurrences
        transition_counts = defaultdict(Counter)
        emission_counts = defaultdict(Counter)

        for s, t in zip(sentences, tags):
            prev_tag = self.start_tag
            for word, tag in zip(s, t):
                transition_counts[prev_tag][tag] += 1
                emission_counts[tag][word] += 1
                self.tag_counts[tag] += 1
                self.tags.add(tag)
                self.vocab.add(word)
                prev_tag = tag
            transition_counts[prev_tag][self.end_tag] += 1

        # Compute probabilities (add-1 smoothing)
        for prev_tag in transition_counts:
            total = sum(transition_counts[prev_tag].values()) + len(self.tags)
            for tag in self.tags.union({self.end_tag}):
                self.transition_probs[prev_tag][tag] = (transition_counts[prev_tag][tag] + 1) / total

        for tag in emission_counts:
            total = sum(emission_counts[tag].values()) + len(self.vocab)
            for word in self.vocab:
                self.emission_probs[tag][word] = (emission_counts[tag][word] + 1) / total

    def viterbi(self, sentence):
        V = [{}]
        path = {}

        # Initialize base cases (start tag)
        for tag in self.tags:
            V[0][tag] = self.transition_probs[self.start_tag][tag]
            path[tag] = [tag]

        # Run Viterbi for t > 0
        for t in range(1, len(sentence)):

```

```

V.append({})
newpath = {}

for curr_tag in self.tags:
    (prob, best_prev_tag) = max(
        (V[t-1][prev_tag] * self.transition_probs[prev_
            self.emission_probs[curr_tag].get(sentence[t],
                for prev_tag in self.tags
        )
    V[t][curr_tag] = prob
    newpath[curr_tag] = path[best_prev_tag] + [curr_tag]
    path = newpath

# Find best final tag
n = len(sentence) - 1
(prob, best_final_tag) = max((V[n][tag], tag) for tag in se
return path[best_final_tag]

```

```

hmm_tagger = HMMPOSTagger()
hmm_tagger.train(train_sents, train_tags)
print("✅ HMM training completed!")

```

✅ HMM training completed!

```

y_true, y_pred = [], []

for s, t in zip(val_sents, val_tags):
    if not s or not t: # skip empty sentences
        continue
    pred = hmm_tagger.viterbi(s)
    y_true.extend(t)
    y_pred.extend(pred)

```

```

from sklearn.metrics import classification_report, accuracy_score

print("✅ HMM POS Tagger Evaluation:")
print("Accuracy:", accuracy_score(y_true, y_pred))
print("\nDetailed Report:\n")
print(classification_report(y_true, y_pred, zero_division=0))

```

✅ HMM POS Tagger Evaluation:
Accuracy: 0.810967150280998

Detailed Report:

	precision	recall	f1-score	support
CC_CCD	0.94	0.93	0.94	923
CC_CCS	0.67	0.70	0.68	281
DM_DMD	0.65	0.90	0.75	784
DM_DMI	0.56	0.33	0.42	66
DM_DMQ	0.00	0.00	0.00	16
DM_DMR	0.20	0.00	0.01	234
JJ	0.67	0.68	0.67	2168
N-NN	0.00	0.00	0.00	1
NN_NNP	0.00	0.00	0.00	1
N_N	0.00	0.00	0.00	1
N_NN	0.79	0.82	0.81	11268
N_NNN	0.00	0.00	0.00	1
N_NNP	0.78	0.67	0.72	4356
N_NP	0.00	0.00	0.00	1
N_NST	0.77	0.43	0.55	510
PR_PRC	0.00	0.00	0.00	1
PR_PRF	0.98	0.90	0.94	155
PR_PRI	0.00	0.00	0.00	49
PR_PRL	0.82	0.72	0.77	329
PR_PRP	0.85	0.80	0.82	463
PR_PRQ	0.50	0.05	0.09	20
PR_PRR	0.00	0.00	0.00	1
PR_RPL	0.00	0.00	0.00	1
PR_RPP	0.00	0.00	0.00	1
PSP	0.89	0.98	0.93	6918
QT_QT	0.00	0.00	0.00	1
QT_QTC	0.91	0.79	0.85	1044
QT_QTF	0.68	0.54	0.60	583
QT_QTO	0.90	0.20	0.32	96
RB	1.00	0.01	0.02	106
RD_ECH	0.00	0.00	0.00	31
RD_PUNC	0.97	0.97	0.97	4024
RD_RDF	0.00	0.00	0.00	9
RD_SYM	0.43	0.28	0.34	58
RP_INJ	0.00	0.00	0.00	3
RP_INTF	0.63	0.68	0.66	112
RP_NEG	0.98	0.93	0.96	129
RP_PRD	0.00	0.00	0.00	1
RP_RPD	0.97	0.86	0.91	761
VM	0.00	0.00	0.00	2
V_VAUX	0.77	0.86	0.81	3323
V_VM	0.67	0.68	0.67	3694
n-nn	0.00	0.00	0.00	1
accuracy			0.81	42527
macro avg	0.44	0.37	0.38	42527
weighted avg	0.81	0.81	0.80	42527

```
for i in range(3):
    print("Sentence:", " ".join(val_sents[i]))
    print("True Tags:", val_tags[i])
    print("Predicted:", hmm_tagger.viterbi(val_sents[i]))
    print("-" * 80)
```

Sentence: इस कला महोत्सव का मुख्य मकसद विभिन्न देशों के कलाविदों के बीच सांस्कृतिक व व
 True Tags: ['DM_DMD', 'N_NN', 'N_NN', 'PSP', 'JJ', 'N_NN', 'JJ', 'N_
 Predicted: ['DM_DMD', 'N_NN', 'N_NN', 'PSP', 'JJ', 'N_NN', 'JJ', 'N_

Sentence: मकाऊ में जेटी से बाहर निकलते ही रिक्शे पर ऊँघते रिक्शा चालकों को देख कर अजीब
 True Tags: ['N_NNP', 'PSP', 'N_NN', 'PSP', 'N_NST', 'V_VM', 'RP_RPD'
 Predicted: ['N_NNP', 'PSP', 'N_NN', 'PSP', 'N_NST', 'V_VM', 'RP_RPD'

Sentence: इसी से मिलती - जुलती स्केटिंग है सिंक्रेनाइज्ड स्केटिंग ।
 True Tags: ['PR_PRL', 'PSP', 'V_VM', 'RD_PUNC', 'RD_ECH', 'N_NN', 'V_
 Predicted: ['DM_DMD', 'PSP', 'V_VM', 'RD_PUNC', 'N_NN', 'V_VM', 'V_

```
import gradio as gr

# Assuming the trained HMM tagger (hmm_tagger) is already available
# If you saved it earlier, you can also load it using pickle.

def tokenize_hindi(text):
    """Simple Hindi tokenizer that splits text by spaces."""
    return text.strip().split()

def predict_hmm_tags(text):
    """Predict POS tags using the trained HMM tagger."""
    tokens = tokenize_hindi(text)
    if not tokens:
        return "कृपया एक वाक्य दर्ज करें।" # Please enter a sentence.
    tags = hmm_tagger.viterbi(tokens)
    return "\n".join(f"{w} / {t}" for w, t in zip(tokens, tags))

# Create Gradio Interface
with gr.Blocks() as demo:
    gr.Markdown("# 🤖 Hindi Hidden Markov Model (HMM) POS Tagger")
    gr.Markdown("Enter a Hindi sentence below to get Part-of-Speech")

    inp = gr.Textbox(lines=4, placeholder="यहाँ हिंदी वाक्य लिखें...")
    out = gr.Textbox(lines=8, label="Predicted Tags")

    btn = gr.Button("Tag Sentence")
    btn.click(fn=predict_hmm_tags, inputs=inp, outputs=out)
```



```
demo.launch(share=True)
```

Colab notebook detected. To show errors in colab notebook, set debug
* Running on public URL: <https://9cf201b923ef4d0139.gradio.live>

This share link expires in 1 week. For free permanent hosting and GP



Hindi Hidden Markov Model (HMM) POS Tagger

Enter a Hindi sentence below to get Part-of-Speech (POS) tags using the HMM approach.

Textbox

यहाँ हिंदी वाक्य लिखें...

Predicted Tags

Conditional Random Field (CRF) POS Tagging

```
!pip install python-crfsuite
```

```
Collecting python-crfsuite
```

```
  Downloading python_crfsuite-0.9.11-cp312-cp312-manylinux_2_17_x86_64
  Downloading python_crfsuite-0.9.11-cp312-cp312-manylinux_2_17_x86_64
```

```
1.3/1.3 MB 56.0 MB/s etc
```

```
Installing collected packages: python-crfsuite
```

```
Successfully installed python-crfsuite-0.9.11
```

```
import pycrfsuite # works the same
```

```
!pip install python-crfsuite pandas indic-nlp-library
```

```
Requirement already satisfied: python-crfsuite in /usr/local/lib/pyt
```

```
Requirement already satisfied: pandas in /usr/local/lib/python3.12/d
```

```
Collecting indic-nlp-library
```

```
  Downloading indic_nlp_library-0.92-py3-none-any.whl.metadata (5.7
```

```
Requirement already satisfied: numpy>=1.26.0 in /usr/local/lib/pytho
```

```
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/
```

```
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python
```

```
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/pyth
```

```
Collecting sphinx-argparse (from indic-nlp-library)
```

```
  Downloading sphinx_argparse-0.5.2-py3-none-any.whl.metadata (3.7 k
```

```
Collecting sphinx-rtd-theme (from indic-nlp-library)
```

```
  Downloading sphinx_rtd_theme-3.0.2-py2.py3-none-any.whl.metadata (
```

```
Collecting morfessor (from indic-nlp-library)
```

```
  Downloading Morfessor-2.0.6-py3-none-any.whl.metadata (628 bytes)
```

```
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12
```

```
Requirement already satisfied: sphinx>=5.1.0 in /usr/local/lib/pytho
```

```
Requirement already satisfied: docutils>=0.19 in /usr/local/lib/pyth
```

```
Collecting sphinxcontrib-jquery<5,>=4 (from sphinx-rtd-theme->indic-
```

```
  Downloading sphinxcontrib_jquery-4.1-py2.py3-none-any.whl.metadata
```

```
Requirement already satisfied: sphinxcontrib-applehelp>=1.0.7 in /us
```

```
Requirement already satisfied: sphinxcontrib-devhelp>=1.0.6 in /usr/
```

```
Requirement already satisfied: sphinxcontrib-htmlhelp>=2.0.6 in /usr
```

```
Requirement already satisfied: sphinxcontrib-jsmath>=1.0.1 in /usr/l
```

```
Requirement already satisfied: sphinxcontrib-qthelp>=1.0.6 in /usr/l
```

```
Requirement already satisfied: sphinxcontrib-serializinghtml>=1.1.9
```

```
Requirement already satisfied: Jinja2>=3.1 in /usr/local/lib/python3
```

```
Requirement already satisfied: Pygments>=2.17 in /usr/local/lib/pyth
```

```
Requirement already satisfied: snowballstemmer>=2.2 in /usr/local/li
```

```
Requirement already satisfied: babel>=2.13 in /usr/local/lib/python3
```

```
Requirement already satisfied: alabaster>=0.7.14 in /usr/local/lib/p
```

```
Requirement already satisfied: imagesize>=1.3 in /usr/local/lib/pyth
```

```
Requirement already satisfied: requests>=2.30.0 in /usr/local/lib/py
```

```
Requirement already satisfied: roman-numerals-py>=1.0.0 in /usr/loca
```

```
Requirement already satisfied: packaging>=23.0 in /usr/local/lib/pyt
```

```
Requirement already satisfied: MarkupSafe>=2.0 in /usr/local/lib/pyt
```

```
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/loca
```

```
Requirement already satisfied: idna<4,>=2.5 in /usr/local/lib/python
```

```
Requirement already satisfied: urllib3<2,>=1.26 in /usr/local/lib/pyt
```

```
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/
Requirement already satisfied: certifi>=2017.4.17 in /usr/local/lib/
Downloading indic_nlp_library-0.92-py3-none-any.whl (40 kB)
 40.3/40.3 kB 3.8 MB/s et
Downloading Morfessor-2.0.6-py3-none-any.whl (35 kB)
Downloading sphinx_argparse-0.5.2-py3-none-any.whl (12 kB)
Downloading sphinx_rtd_theme-3.0.2-py2.py3-none-any.whl (7.7 MB)
 7.7/7.7 MB 105.7 MB/s et
Downloading sphinxcontrib_jquery-4.1-py2.py3-none-any.whl (121 kB)
 121.1/121.1 kB 11.5 MB/s
Installing collected packages: morfessor, sphinxcontrib-jquery, sphi
Successfully installed indic-nlp-library-0.92 morfessor-2.0.6 sphinx
```

```

# Colab cell
import os
import codecs

data_folder = "/content/drive/MyDrive/HINDI_DATASET" # adjust if n

def parse_line(line):
    # Example line: htd1\tसप्तपुरियों\N_NNP के\PSP दर्शन\N_NN ...
    parts = line.strip().split("\t", 1)
    if len(parts) < 2:
        return None
    seq = []
    for token in parts[1].split():
        if "\\\" in token:
            word, tag = token.split("\\\", 1)
            seq.append((word, tag))
    return seq if seq else None

sentences = []
file_count = 0
for fname in sorted(os.listdir(data_folder)):
    if not fname.lower().endswith(".txt"):
        continue
    file_count += 1
    path = os.path.join(data_folder, fname)
    with codecs.open(path, 'r', 'utf-8') as f:
        for line in f:
            parsed = parse_line(line)
            if parsed:
                sentences.append(parsed)

print(f"Loaded {len(sentences)} sentences from {file_count} files.")
print("Example:", sentences[0][:20])

```

Loaded 25000 sentences from 25 files.
 Example: [('सप्तपुरियों', 'N_NNP'), ('के', 'PSP'), ('दर्शन', 'N_NN'), ('से',

```

# Colab cell
import re

PUNCT_RE = re.compile(r'^[\u0964\u0021-\u002F\u003A-\u0040\u005B-\u

def word_shape(w):
    # simple shape: all digits, contains digit, upper/lower not mea
    if w.isdigit():

```

```

        return "digits"
    if PUNCT_RE.match(w):
        return "punct"
    return "other"

def word2features(sent, i):
    word = sent[i][0]
    f = {
        'bias': 1.0,
        'word': word,
        'word.lower()': word.lower(),
        'word.isdigit()': word.isdigit(),
        'word.shape': word_shape(word),
        'word.prefix1': word[:1],
        'word.prefix2': word[:2] if len(word) > 1 else word,
        'word.prefix3': word[:3] if len(word) > 2 else word,
        'word.suffix1': word[-1:],
        'word.suffix2': word[-2:] if len(word) > 1 else word,
        'word.suffix3': word[-3:] if len(word) > 2 else word,
        'word.len': len(word),
        'BOS': i == 0,
        'EOS': i == len(sent)-1,
    }
    if i > 0:
        prevw = sent[i-1][0]
        f.update({
            '-1:word': prevw,
            '-1:word.lower()': prevw.lower(),
            '-1:shape': word_shape(prevw),
        })
    else:
        f['BOS_token'] = True
    if i < len(sent)-1:
        nextw = sent[i+1][0]
        f.update({
            '+1:word': nextw,
            '+1:word.lower()': nextw.lower(),
            '+1:shape': word_shape(nextw),
        })
    else:
        f['EOS_token'] = True
    return f

def sent2features(sent):
    return [word2features(sent, i) for i in range(len(sent))]

def sent2labels(sent):
    return [tag for (_, tag) in sent]

```

```
def sent2tokens(sent):
    return [w for (w, _) in sent]
```

```
# Colab cell
from sklearn.model_selection import train_test_split

train_sents, test_sents = train_test_split(sentences, test_size=0.2)

X_train = [sent2features(s) for s in train_sents]
y_train = [sent2labels(s) for s in train_sents]

X_test = [sent2features(s) for s in test_sents]
y_test = [sent2labels(s) for s in test_sents]

print("Train sentences:", len(X_train))
print("Test sentences:", len(X_test))
```

```
Train sentences: 20000
Test sentences: 5000
```

```
# Colab cell
import pycrfsuite
trainer = pycrfsuite.Trainer(verbose=True)

for xseq, yseq in zip(X_train, y_train):
    trainer.append(xseq, yseq)

trainer.set_params({
    'c1': 0.1,    # L1
    'c2': 0.01,  # L2
    'max_iterations': 200,
    'feature.possible_transitions': True
})

model_path = '/content/drive/MyDrive/hindi_pos_crf.model'
trainer.train(model_path)
print("Saved model to:", model_path)
```

```
Feature generation
type: CRF1d
feature.minfreq: 0.000000
feature.possible_states: 0
feature.possible_transitions: 1
```

0....1....2....3....4....5....6....7....8....9....10

Number of features: 344824

Seconds required: 2.296

L-BFGS optimization

c1: 0.100000

c2: 0.010000

num_memories: 6

max_iterations: 200

epsilon: 0.000010

stop: 10

delta: 0.000010

linesearch: MoreThuente

linesearch.max_iterations: 20

***** Iteration #1 *****

Loss: 1139384.247284

Feature norm: 1.000000

Error norm: 335333.310960

Active features: 342918

Line search trials: 1

Line search step: 0.000002

Seconds required for this iteration: 20.692

***** Iteration #2 *****

Loss: 956044.912054

Feature norm: 1.605087

Error norm: 292868.715745

Active features: 335218

Line search trials: 1

Line search step: 1.000000

Seconds required for this iteration: 9.685

***** Iteration #3 *****

Loss: 830142.640794

Feature norm: 2.499373

Error norm: 263241.349963

Active features: 338658

Line search trials: 1

Line search step: 1.000000

Seconds required for this iteration: 10.440

***** Iteration #4 *****

Loss: 753902.195637

Feature norm: 3.131288

Error norm: 266302.884103

Active features: 339840

Line search trials: 1

Line search step: 1.000000

Seconds required for this iteration: 10.313

***** Iteration #5 *****

Loss: 697604.100564

```
import pycrfsuite

model_path = '/content/drive/MyDrive/hindi_pos_crf.model'
tagger = pycrfsuite.Tagger()
tagger.open(model_path)

# Now you can predict
y_pred = [tagger.tag(xseq) for xseq in X_test]
```

```
# Colab cell
tagger = pycrfsuite.Tagger()
tagger.open(model_path)

# Predict on test set (sequence-level)
y_pred = [tagger.tag(xseq) for xseq in X_test]
```

```
# Colab cell
from sklearn.metrics import classification_report
import itertools

y_true_flat = list(itertools.chain.from_iterable(y_test))
y_pred_flat = list(itertools.chain.from_iterable(y_pred))

# Print overall classification report
print(classification_report(y_true_flat, y_pred_flat, digits=4))
```

NNP	0.0000	0.0000	0.0000	0
N_	0.0000	0.0000	0.0000	1
N_N	0.0000	0.0000	0.0000	1
N_NJN	0.0000	0.0000	0.0000	1
N_NN	0.8454	0.8538	0.8496	22216
N_NNN	0.0000	0.0000	0.0000	1
N_NNP	0.8377	0.8272	0.8324	8877
N_NNp	0.0000	0.0000	0.0000	2
N_NP	0.0000	0.0000	0.0000	0
N_NST	0.6928	0.6995	0.6961	935
N_N_NNP	0.0000	0.0000	0.0000	2
PPS	0.0000	0.0000	0.0000	1
PR_PRC	0.6250	0.8333	0.7143	6
PR_PRF	0.9861	0.9373	0.9611	303
PR_PRI	0.2439	0.0962	0.1379	104
PR_PRL	0.7637	0.6980	0.7294	639
PR_PRP	0.8233	0.8618	0.8421	854
PR_PRQ	0.6471	0.6471	0.6471	34
PR_RPL	0.0000	0.0000	0.0000	1

PSP	0.9722	0.9802	0.9762	13776
QT-QTO	0.0000	0.0000	0.0000	1
QT_QTC	0.9486	0.9477	0.9482	2105
QT_QTF	0.6599	0.6515	0.6557	1099
QT_QTO	0.7078	0.6853	0.6964	251
QT_QT_QTC	0.0000	0.0000	0.0000	1
QT_TQC	0.0000	0.0000	0.0000	1
RB	0.2832	0.1616	0.2058	198
RD_ECH	0.4400	0.2200	0.2933	50
RD_PUNC	0.9870	0.9901	0.9886	7912
RD_PUNCS	0.0000	0.0000	0.0000	1
RD_RDF	0.9091	0.6250	0.7407	32
RD_SYM	0.6316	0.7143	0.6704	168
RP_INJ	0.5000	0.1250	0.2000	8
RP_INTF	0.6232	0.7107	0.6641	242
RP_NEG	0.9849	0.9775	0.9812	267
RP_PRD	0.0000	0.0000	0.0000	2
RP_RP	0.0000	0.0000	0.0000	1
RP_RPD	0.9450	0.8868	0.9150	1511
SPP	0.0000	0.0000	0.0000	1
V_VAUX	0.8878	0.8326	0.8593	6565
V_VAUX\RD_PUNC	0.0000	0.0000	0.0000	1
V_VM	0.7680	0.8233	0.7947	7312
V_VVAUX	0.0000	0.0000	0.0000	1
\N_NNP	0.0000	0.0000	0.0000	1
_NNP	0.0000	0.0000	0.0000	1
_VAUX	0.0000	0.0000	0.0000	1
accuracy			0.8635	84323
macro avg	0.3889	0.3719	0.3754	84323
weighted avg	0.8620	0.8635	0.8622	84323

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(re
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(re
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification
_warn_prf(average, modifier, f"{metric.capitalize()} is", len(re
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification
warn_prf(average, modifier, f"{metric.capitalize()} is", len(re

```

```

# Colab cell
from sklearn.metrics import classification_report
import itertools

y_true_flat = list(itertools.chain.from_iterable(y_test))
y_pred_flat = list(itertools.chain.from_iterable(y_pred))

# Print overall classification report
print(classification_report(y_true_flat, y_pred_flat, digits=4))

```

```

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1258: _warn_prf(average, modifier, f"{metric.capitalize()} is", len(re
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1258: _warn_prf(average, modifier, f"{metric.capitalize()} is", len(re
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1258: _warn_prf(average, modifier, f"{metric.capitalize()} is", len(re
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1258: _warn_prf(average, modifier, f"{metric.capitalize()} is", len(re
/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1258: _warn_prf(average, modifier, f"{metric.capitalize()} is", len(re

```

	precision	recall	f1-score	support
	0.0000	0.0000	0.0000	1
CC_CCD	0.9406	0.9523	0.9464	1845
CC_CCS	0.7258	0.7104	0.7180	518
DD_DMD	0.0000	0.0000	0.0000	1
DM_DM	0.0000	0.0000	0.0000	1
DM_DMD	0.7116	0.8425	0.7715	1517
DM_DMI	0.5156	0.6226	0.5641	106
DM_DMQ	0.4231	0.3333	0.3729	33
DM_DMR	0.3894	0.2369	0.2946	498
DR_DMD	0.0000	0.0000	0.0000	1
JJ	0.7465	0.7152	0.7305	4315
NNP	0.0000	0.0000	0.0000	0
N_	0.0000	0.0000	0.0000	1
N_N	0.0000	0.0000	0.0000	1
N_NJN	0.0000	0.0000	0.0000	1
N_NN	0.8454	0.8538	0.8496	22216
N_NNN	0.0000	0.0000	0.0000	1
N_NNP	0.8377	0.8272	0.8324	8877
N_NNp	0.0000	0.0000	0.0000	2
N_NP	0.0000	0.0000	0.0000	0
N_NST	0.6928	0.6995	0.6961	935
N_N_NNP	0.0000	0.0000	0.0000	2
PPS	0.0000	0.0000	0.0000	1
PR_PRC	0.6250	0.8333	0.7143	6
PR_PRF	0.9861	0.9373	0.9611	303
PR_PRI	0.2439	0.0962	0.1379	104
PR_PRL	0.7637	0.6980	0.7294	639
PR_PRP	0.8233	0.8618	0.8421	854
PR_PRQ	0.6471	0.6471	0.6471	34
PR_RPL	0.0000	0.0000	0.0000	1
PSP	0.9722	0.9802	0.9762	13776
QT-QT0	0.0000	0.0000	0.0000	1
QT_QTC	0.9486	0.9477	0.9482	2105
QT_QTF	0.6599	0.6515	0.6557	1099
QT_QT0	0.7078	0.6853	0.6964	251
QT_QT_QTC	0.0000	0.0000	0.0000	1
QT_TQC	0.0000	0.0000	0.0000	1
RB	0.2832	0.1616	0.2058	198
RD_ECH	0.4400	0.2200	0.2933	50
RD_PUNC	0.9870	0.9901	0.9886	7912
RD_PUNCS	0.0000	0.0000	0.0000	1
RD_RDF	0.9091	0.6250	0.7407	32
RD_SYM	0.6316	0.7143	0.6704	168
RP_INJ	0.5000	0.1250	0.2000	8

RP_INTF	0.6232	0.7107	0.6641	242
RP_NEG	0.9849	0.9775	0.9812	267
RP_PRD	0.0000	0.0000	0.0000	2
RP_RP	0.0000	0.0000	0.0000	1
RP_RPN	0.9450	0.8868	0.9150	1511

```
# Colab cell
from sklearn.metrics import confusion_matrix
import numpy as np
import pandas as pd

labels = sorted(list(set(y_true_flat) | set(y_pred_flat)))
cm = confusion_matrix(y_true_flat, y_pred_flat, labels=labels)
cm_df = pd.DataFrame(cm, index=labels, columns=labels)
# Show top confusions (largest off-diagonal entries)
cm_df.to_csv('/content/drive/MyDrive/crf_confusion_matrix.csv', encoding='utf-8')
print("Confusion matrix saved to Drive.")
cm_df.head(10)
```

Confusion matrix saved to Drive.

		CC_CCD	CC_CCS	DD_DMD	DM_DM	DM_DMD	DM_DMI	DM_DMQ	DM_DMR	DR_DMD
	0	0	0	0	0	0	0	0	0	0
CC_CCD	0	1757	44	0	0	2	0	0	0	0
CC_CCS	0	45	368	0	0	4	0	0	0	0
DD_DMD	0	0	0	0	0	0	0	0	0	0
DM_DM	0	0	0	0	0	1	0	0	0	0
DM_DMD	0	0	4	0	0	1278	0	0	0	0
DM_DMI	0	0	1	0	0	3	66	4	0	0
DM_DMQ	0	0	0	0	0	0	7	11	0	0
DM_DMR	0	0	4	0	0	273	2	0	0	0
DR_DMD	0	0	0	0	0	1	0	0	0	0

10 rows × 57 columns

```
# Example function for predicting a new Hindi sentence
def tag_sentence(sentence):
    # naive tokenization
    tokens = sentence.strip().split()
    sent = [(t, None) for t in tokens] # convert to (word, None)
    feats = sent2features(sent) # use your feature extractor
    tags = tagger.tag(feats)
    return list(zip(tokens, tags))

# Example
tag_sentence("राम घर में खेलता है ।")
```

```
[('राम', 'N_NNP'),
 ('घर', 'N_NN'),
 ('में', 'PSP'),
 ('खेलता', 'V_VM'),
 ('है', 'V_VAUX'),
 ('।', 'RD_PUNC')]
```

```
!pip install gradio
import gradio as gr

def gradio_tagger(sentence):
    tokens = sentence.strip().split()
    sent = [(t, None) for t in tokens]
    feats = sent2features(sent)
    tags = tagger.tag(feats)
    return " ".join([f"{w}/{t}" for w,t in zip(tokens, tags)])

iface = gr.Interface(fn=gradio_tagger,
                     inputs=gr.Textbox(lines=3, placeholder="हिंदी वाक्य",
                                       outputs="text",
                                       title="Hindi POS Tagger (CRF)")
                     )

iface.launch()
```

```
Requirement already satisfied: gradio in /usr/local/lib/python3.12/d
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/li
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/pyt
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/pytho
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local/l
Requirement already satisfied: ffmpeg in /usr/local/lib/python3.12/di
Requirement already satisfied: gradio-client==1.13.3 in /usr/local/l
Requirement already satisfied: groovy~=0.1 in /usr/local/lib/python3
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/lib/
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /usr/
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python3.
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/li
```

```
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/pyt
Requirement already satisfied: orjson~=3.0 in /usr/local/lib/python3
Requirement already satisfied: packaging in /usr/local/lib/python3.1
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/py
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib/p
Requirement already satisfied: pydantic<2.12,>=2.0 in /usr/local/lib
Requirement already satisfied: pydub in /usr/local/lib/python3.12/di
Requirement already satisfied: python-multipart>=0.0.18 in /usr/loca
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/py
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/python3
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/local
Requirement already satisfied: semantic-version~=2.0 in /usr/local/l
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/local/
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/local
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/py
Requirement already satisfied: typing-extensions~=4.0 in /usr/local/
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/pyt
Requirement already satisfied: fsspec in /usr/local/lib/python3.12/d
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/local/
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3.1
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/python
Requirement already satisfied: certifi in /usr/local/lib/python3.12/
Requirement already satisfied: httpcore==1.* in /usr/local/lib/pytho
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3.1
Requirement already satisfied: filelock in /usr/local/lib/python3.12
Requirement already satisfied: requests in /usr/local/lib/python3.12
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/python
Requirement already satisfied: hf-xet<2.0.0,>=1.1.3 in /usr/local/li
Requirement already satisfied: python-dateutil>=2.8.2 in /usr/local/
Requirement already satisfied: pytz>=2020.1 in /usr/local/lib/python
Requirement already satisfied: tzdata>=2022.7 in /usr/local/lib/pyth
Requirement already satisfied: annotated-types>=0.6.0 in /usr/local/
Requirement already satisfied: pydantic-core==2.33.2 in /usr/local/l
Requirement already satisfied: typing-inspection>=0.4.0 in /usr/loca
Requirement already satisfied: click>=8.0.0 in /usr/local/lib/python
Requirement already satisfied: shellingham>=1.3.0 in /usr/local/lib/
Requirement already satisfied: rich>=10.11.0 in /usr/local/lib/pytho
Requirement already satisfied: six>=1.5 in /usr/local/lib/python3.12
Requirement already satisfied: markdown-it-py>=2.2.0 in /usr/local/l
Requirement already satisfied: pygments<3.0.0,>=2.13.0 in /usr/local
Requirement already satisfied: charset_normalizer<4,>=2 in /usr/loca
Requirement already satisfied: urllib3<3,>=1.21.1 in /usr/local/lib/
Requirement already satisfied: mdurl~=0.1 in /usr/local/lib/python3.
It looks like you are running Gradio on a hosted Jupyter notebook, w
```

Colab notebook detected. To show errors in colab notebook, set debug
 * Running on public URL: <https://ad5c47fd47821ddc3e.gradio.live>

This share link expires in 1 week. For free permanent hosting and GP



No interface is running right now

BiLSTM POS Tagger (Deep Learning approach)

```
import numpy as np
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import LSTM, Dense, Embedding, TimeDis
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
```

```
from sklearn.model_selection import train_test_split

# Separate words and tags from your parsed sentences
sentences_words = [[w for w, t in s] for s in sentences]
sentences_tags = [[t for w, t in s] for s in sentences]

# Split into training and validation
train_sents, val_sents, train_tags, val_tags = train_test_split(
    sentences_words, sentences_tags, test_size=0.2, random_state=42
)

print(f"✅ Training sentences: {len(train_sents)}")
print(f"✅ Validation sentences: {len(val_sents)}")
```

```
✅ Training sentences: 20000
✅ Validation sentences: 5000
```

```
# Build vocab from all data (train + validation)
all_tags = [t for tag_seq in (train_tags + val_tags) for t in tag_seq]
all_words = [w for sent in (train_sents + val_sents) for w in sent]

word2idx = {w: i + 2 for i, w in enumerate(set(all_words))}
word2idx["PAD"] = 0
word2idx["UNK"] = 1

tag2idx = {t: i + 1 for i, t in enumerate(set(all_tags))}
tag2idx["PAD"] = 0

print(f"✅ Total unique words:", len(word2idx))
print(f"✅ Total unique tags:", len(tag2idx))
```

```
✅ Total unique words: 29353
✅ Total unique tags: 113
```

```
# Build word and tag vocabularies
words = list(set([w for sent in train_sents for w in sent]))
tags = list(set([t for tag_seq in train_tags for t in tag_seq]))

word2idx = {w: i + 2 for i, w in enumerate(words)}
word2idx["PAD"] = 0
word2idx["UNK"] = 1

tag2idx = {t: i + 1 for i, t in enumerate(tags)}
tag2idx["PAD"] = 0
```

```
from tensorflow.keras.preprocessing.sequence import pad_sequences
from tensorflow.keras.utils import to_categorical
import numpy as np

# Convert words and tags to indices
X_train = [[word2idx.get(w, word2idx["UNK"]) for w in s] for s in train_sents]
X_val = [[word2idx.get(w, word2idx["UNK"]) for w in s] for s in val_sents]

y_train = [[tag2idx.get(t, tag2idx["PAD"]) for t in s] for s in train_tags]
y_val = [[tag2idx.get(t, tag2idx["PAD"]) for t in s] for s in val_tags]

# Pad sequences
MAX_LEN = max(len(s) for s in train_sents)
X_train = pad_sequences(X_train, maxlen=MAX_LEN, padding='post')
X_val = pad_sequences(X_val, maxlen=MAX_LEN, padding='post')
y_train = pad_sequences(y_train, maxlen=MAX_LEN, padding='post')

# Convert tags to categorical for training
y_train = to_categorical(y_train, num_classes=len(tag2idx))
```



```

from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, Bidirectional, LSTM,

model = Sequential()
model.add(Embedding(input_dim=len(word2idx), output_dim=128, input_
model.add(Bidirectional(LSTM(units=64, return_sequences=True, recur
model.add(TimeDistributed(Dense(len(tag2idx), activation="softmax"))

model.compile(optimizer='adam', loss='categorical_crossentropy', me
model.summary()

```

```

/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/embedd
warnings.warn(
Model: "sequential"

```

Layer (type)	Output Shape	F
embedding (Embedding)	?	0 (ur
bidirectional (Bidirectional)	?	0 (ur
time_distributed (TimeDistributed)	?	0 (ur

Total params: 0 (0.00 B)
 Trainable params: 0 (0.00 B)
 Non-trainable params: 0 (0.00 B)

```

history = model.fit(
    X_train, y_train,
    batch_size=32,
    epochs=3,
    validation_split=0.1,
    verbose=1
)

```

```

Epoch 1/3
563/563 ————— 217s 363ms/step - accuracy: 0.1152 - lc
Epoch 2/3
563/563 ————— 194s 345ms/step - accuracy: 0.1728 - lc
Epoch 3/3
563/563 ————— 189s 335ms/step - accuracy: 0.1766 - lc

```

```

from tensorflow.keras.preprocessing.sequence import pad_sequences
import numpy as np

# ◆ Rebuild tag vocabulary safely using ALL tags (train + val)
all_tags = sorted(list(set([t for seq in (train_tags + val_tags) for t in seq])))
tag2idx = {t: i + 1 for i, t in enumerate(all_tags)}
tag2idx["PAD"] = 0
idx2tag = {v: k for k, v in tag2idx.items()}

print("✅ Total tags:", len(tag2idx))
print("Example tags:", list(tag2idx.keys())[:10])

# ◆ Convert predictions to tag indices
y_pred_tags = np.argmax(y_pred, axis=-1)

# ◆ Convert true tags safely (replace missing tags with PAD)
y_true_tags = np.array([
    pad_sequences(
        [[tag2idx.get(t, tag2idx["PAD"]) for t in s]],
        maxlen=MAX_LEN, padding='post'
    )[0]
    for s in val_tags
])

# ◆ Mask PAD positions and compute accuracy
mask = y_true_tags != tag2idx["PAD"]
accuracy = (y_pred_tags[mask] == y_true_tags[mask]).mean()

print(f"\n✅ BiLSTM Validation Accuracy: {accuracy:.4f}")

```

```

✅ Total tags: 113
Example tags: ['', 'CCC_CD', 'CC_CCD', 'CC_CCS', 'CC_CS', 'C_CCD', 'C_CCS', 'C_CD', 'C_C', 'C']

✅ BiLSTM Validation Accuracy: 0.0009

```

```

for i in range(3):
    sent = val_sents[i]
    X_sample = pad_sequences([[word2idx.get(w, word2idx["UNK"]) for
                               maxlen=MAX_LEN, padding='post')
    pred = np.argmax(model.predict(X_sample), axis=-1)[0]
    print("👉 Sentence:", " ".join(sent))
    print("Predicted Tags:", [idx2tag[p] for p in pred[:len(sent)]]
    print("-" * 80)

```

1/1  0s 143ms/step

👉 Sentence: दमा , श्वास , हृदय और रक्तचाप वाले रोगियों के लिये यह चढ़ाई चढ़ना मना है
 Predicted Tags: ['PR_PRL', 'NN', 'PR_PRL', 'NN', 'RP', 'V_VAUXI\\RD_

1/1  0s 209ms/step

👉 Sentence: ट्रेकिंग के शौकीन लोगों को यह ग्लेशियर बहुत आकर्षित करते हैं ।
 Predicted Tags: ['RP', 'N_N_NNP', 'V_V', 'RP', 'N_N_NNP', 'RD_PUNC',

1/1  0s 179ms/step

👉 Sentence: नेओरा घाटी हमारे देश की उन वन-संपदाओं में से है जिनकी प्राकृतिकता अभी तक
 Predicted Tags: ['PR_PRL', 'PR_PRL', 'DM_DMR', 'RP', 'N_N_NNP', 'DM_

```

model.save('/content/drive/MyDrive/hindi_pos_bilstm.h5')
print("✅ Model saved to Google Drive successfully!")

```

WARNING:absl:You are saving your model as an HDF5 file via `model.save`
 ✅ Model saved to Google Drive successfully!

```
!pip install gradio tensorflow
```

```

Requirement already satisfied: gradio in /usr/local/lib/python3.12/
Requirement already satisfied: tensorflow in /usr/local/lib/python3.12/
Requirement already satisfied: aiofiles<25.0,>=22.0 in /usr/local/
Requirement already satisfied: anyio<5.0,>=3.0 in /usr/local/lib/p
Requirement already satisfied: brotli>=1.1.0 in /usr/local/lib/pytl
Requirement already satisfied: fastapi<1.0,>=0.115.2 in /usr/local
Requirement already satisfied: ffmpeg in /usr/local/lib/python3.12/
Requirement already satisfied: gradio-client==1.13.3 in /usr/local
Requirement already satisfied: groovy~0.1 in /usr/local/lib/pytho
Requirement already satisfied: httpx<1.0,>=0.24.1 in /usr/local/li
Requirement already satisfied: huggingface-hub<2.0,>=0.33.5 in /us
Requirement already satisfied: jinja2<4.0 in /usr/local/lib/python
Requirement already satisfied: markupsafe<4.0,>=2.0 in /usr/local/
Requirement already satisfied: numpy<3.0,>=1.0 in /usr/local/lib/p
Requirement already satisfied: orjson~3.0 in /usr/local/lib/pytho

```

```

Requirement already satisfied: packaging in /usr/local/lib/python3
Requirement already satisfied: pandas<3.0,>=1.0 in /usr/local/lib/
Requirement already satisfied: pillow<12.0,>=8.0 in /usr/local/lib
Requirement already satisfied: pydantic<2.12,>=2.0 in /usr/local/l
Requirement already satisfied: pydub in /usr/local/lib/python3.12/
Requirement already satisfied: python-multipart>=0.0.18 in /usr/lo
Requirement already satisfied: pyyaml<7.0,>=5.0 in /usr/local/lib/
Requirement already satisfied: ruff>=0.9.3 in /usr/local/lib/pytho
Requirement already satisfied: safehttpx<0.2.0,>=0.1.6 in /usr/loc
Requirement already satisfied: semantic-version~=2.0 in /usr/local
Requirement already satisfied: starlette<1.0,>=0.40.0 in /usr/loca
Requirement already satisfied: tomlkit<0.14.0,>=0.12.0 in /usr/loc
Requirement already satisfied: typer<1.0,>=0.12 in /usr/local/lib/
Requirement already satisfied: typing-extensions~=4.0 in /usr/loca
Requirement already satisfied: uvicorn>=0.14.0 in /usr/local/lib/p
Requirement already satisfied: fsspec in /usr/local/lib/python3.12
Requirement already satisfied: websockets<16.0,>=13.0 in /usr/loca
Requirement already satisfied: absl-py>=1.0.0 in /usr/local/lib/py
Requirement already satisfied: astunparse>=1.6.0 in /usr/local/lib
Requirement already satisfied: flatbuffers>=24.3.25 in /usr/local/
Requirement already satisfied: gast!=0.5.0,!0.5.1,!0.5.2,>=0.2.1
Requirement already satisfied: google-pasta>=0.1.1 in /usr/local/l
Requirement already satisfied: libclang>=13.0.0 in /usr/local/lib/
Requirement already satisfied: opt-einsum>=2.3.2 in /usr/local/lib
Requirement already satisfied: protobuf!=4.21.0,!4.21.1,!4.21.2,
Requirement already satisfied: requests<3,>=2.21.0 in /usr/local/l
Requirement already satisfied: setuptools in /usr/local/lib/python
Requirement already satisfied: six>=1.12.0 in /usr/local/lib/pytho
Requirement already satisfied: termcolor>=1.1.0 in /usr/local/lib/
Requirement already satisfied: wrapt>=1.11.0 in /usr/local/lib/pytl
Requirement already satisfied: grpcio<2.0,>=1.24.3 in /usr/local/l
Requirement already satisfied: tensorboard~=2.19.0 in /usr/local/l
Requirement already satisfied: keras>=3.5.0 in /usr/local/lib/pyth
Requirement already satisfied: h5py>=3.11.0 in /usr/local/lib/pyth
Requirement already satisfied: ml-dtypes<1.0.0,>=0.5.1 in /usr/loc
Requirement already satisfied: idna>=2.8 in /usr/local/lib/python3
Requirement already satisfied: sniffio>=1.1 in /usr/local/lib/pyth
Requirement already satisfied: wheel<1.0,>=0.23.0 in /usr/local/li
Requirement already satisfied: certifi in /usr/local/lib/python3.1
Requirement already satisfied: httpcore==1.* in /usr/local/lib/pytl
Requirement already satisfied: h11>=0.16 in /usr/local/lib/python3
Requirement already satisfied: filelock in /usr/local/lib/python3.
Requirement already satisfied: tqdm>=4.42.1 in /usr/local/lib/pyth
Requirement already satisfied: hf-vet<2.0.0,>=1.1.2 in /usr/local/

```

```
import pickle

with open('/content/drive/MyDrive/word2idx.pkl', 'wb') as f:
    pickle.dump(word2idx, f)

with open('/content/drive/MyDrive/idx2tag.pkl', 'wb') as f:
    pickle.dump(idx2tag, f)
```

```
from tensorflow.keras.models import load_model
import pickle
import numpy as np
from tensorflow.keras.preprocessing.sequence import pad_sequences

# Load BiLSTM model from Drive
model = load_model('/content/drive/MyDrive/hindi_pos_bilstm.h5')

# Load word and tag mappings
with open('/content/drive/MyDrive/word2idx.pkl', 'rb') as f:
    word2idx = pickle.load(f)
with open('/content/drive/MyDrive/idx2tag.pkl', 'rb') as f:
    idx2tag = pickle.load(f)

MAX_LEN = 100 # same as used during training
```

WARNING:absl:Compiled the loaded model, but the compiled metrics have

```
def predict_pos(sentence):
    tokens = sentence.strip().split()
    seq = [word2idx.get(w, word2idx.get("UNK", 1)) for w in tokens]
    seq_padded = pad_sequences([seq], maxlen=MAX_LEN, padding='post')

    pred = model.predict(seq_padded)
    pred_tags = np.argmax(pred, axis=-1)[0][:len(tokens)]

    tags = [idx2tag.get(i, "UNK") for i in pred_tags]
    return " ".join([f"{w}/{t}" for w, t in zip(tokens, tags)])
```

```
import gradio as gr

iface = gr.Interface(
    fn=predict_pos,
    inputs=gr.Textbox(lines=3, placeholder="हिंदी वाक्य लिखें..."),
    outputs="text",
    title="🦉 Hindi POS Tagger (BiLSTM Model)",
    description="Enter a Hindi sentence to see its Part-of-Speech tags"
)

iface.launch(share=True) # share=True gives a public test URL
```

Colab notebook detected. To show errors in colab notebook, set debug
* Running on public URL: <https://8f57e96b1b61cf58be.gradio.live>

This share link expires in 1 week. For free permanent hosting and GPU



No interface is running right now

Transformer-Based Model (mBERT)

```

from google.colab import drive
drive.mount('/content/drive')

import os

# ✅ Change only this if your folder name is different
DATA_DIR = "/content/drive/MyDrive/HINDI_DATASET"

# List files to confirm
print("📁 Files in dataset directory:")
if os.path.exists(DATA_DIR):
    print(os.listdir(DATA_DIR))
else:
    print("❌ Folder not found. Check path.")

```

```

Mounted at /content/drive
📁 Files in dataset directory:
['hin_tourism_set04.txt', 'hin_tourism_set08.txt', 'hin_tourism_set1

```

```

# =====
# ✅ CONFIGURATION
# =====
MODEL_NAME = "bert-base-multilingual-cased"
MAX_LEN = 128
BATCH_SIZE = 8
EPOCHS = 3
LR = 3e-5

import torch
DEVICE = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print("✅ Using device:", DEVICE)

```

```

✅ Using device: cuda

```

```

# =====
# 📌 STEP 1: Load Dataset
# =====
import glob, re

files = sorted(glob.glob(DATA_DIR + "/*.txt"))
print(f"📁 Total files found: {len(files)}")
if len(files) == 0:
    raise Exception("❌ No .txt files found. Please check dataset 1

def load_data(files):
    sentences, labels = [], []
    for file in files:
        with open(file, 'r', encoding='utf-8') as f:
            for line in f:
                line = line.strip()
                if not line: continue
                line = re.sub(r'^\s*\d+\s*', '', line) # remove
                words = []
                tags = []
                for token in line.split():
                    if "\\\" in token:
                        word, tag = token.split("\\\", 1)
                        words.append(word)
                        tags.append(tag)
                if words:
                    sentences.append(words)
                    labels.append(tags)
    return sentences, labels

sentences, labels = load_data(files)
print(f"✅ Total sentences loaded:", len(sentences))
print(f"📝 Example:", sentences[0][:5], labels[0][:5])

```

📁 Total files found: 25
 ✅ Total sentences loaded: 25000
 📝 Example: ['सप्तपुरियों', 'के', 'दर्शन', 'से', 'मिलता'] ['N_NNP', 'PSP', 'N


```
# =====
# 📌 STEP 2: Tokenizer & Labels
# =====
from transformers import AutoTokenizer
tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME, use_fast=True)

unique_tags = sorted(set(tag for seq in labels for tag in seq))
label2id = {tag: i for i, tag in enumerate(unique_tags)}
id2label = {i: tag for tag, i in label2id.items()}
num_labels = len(unique_tags)
print("✅ Total Tags:", num_labels, unique_tags)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your se
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to
warnings.warn(
```

```
tokenizer_config.json: 100% 49.0/49.0 [00:00<00:00, 2.45kB/s]
config.json: 100% 625/625 [00:00<00:00, 8.67kB/s]
vocab.txt: 100% 996k/996k [00:00<00:00, 4.89MB/s]
tokenizer.json: 100% 1.96M/1.96M [00:00<00:00, 6.91MB/s]
```

```
✅ Total Tags: 112 [' ', 'CCC_CD', 'CC_CCD', 'CC_CCS', 'CC_CS', 'C_CC
```

```
# =====
# 📌 STEP 3: Dataset Class
# =====
from torch.utils.data import Dataset, DataLoader

class PosDataset(Dataset):
    def __init__(self, sentences, labels):
        self.sentences = sentences
        self.labels = labels

    def __len__(self):
        return len(self.sentences)

    def __getitem__(self, idx):
        words = self.sentences[idx]
        labs = self.labels[idx]
        encoding = tokenizer(words, is_split_into_words=True, trunc
                               max_length=MAX_LEN, return_tensors='pt')
        word_ids = encoding.word_ids()
        label_ids = []
        for word_idx in word_ids:
            if word_idx is None:
                label_ids.append(-100)
            else:
                label_ids.append(label2id[labs[word_idx]])
        encoding["labels"] = torch.tensor(label_ids)
        return {key: val.squeeze().to(DEVICE) for key, val in encoding.items()}

# Split dataset
from sklearn.model_selection import train_test_split
train_s, test_s, train_l, test_l = train_test_split(sentences, labels,
                                                    train_size=0.8,
                                                    random_state=42)

train_ds = PosDataset(train_s, train_l)
test_ds = PosDataset(test_s, test_l)

train_loader = DataLoader(train_ds, batch_size=BATCH_SIZE, shuffle=True)
test_loader = DataLoader(test_ds, batch_size=BATCH_SIZE)
print("✅ DataLoaders ready!")
```

✅ DataLoaders ready!

```
# =====
# 📌 STEP 4: Load Model
# =====
from transformers import AutoModelForTokenClassification, get_linear_schedule_with_warmup
from torch.optim import AdamW

model = AutoModelForTokenClassification.from_pretrained(
    MODEL_NAME, num_labels=num_labels, id2label=id2label, label2id=
).to(DEVICE)

optimizer = AdamW(model.parameters(), lr=LR)
total_steps = len(train_loader) * EPOCHS
scheduler = get_linear_schedule_with_warmup(optimizer,
                                             num_warmup_steps=int(0.
                                             num_training_steps=tota
```

model.safetensors: 100%

714M/714M [00:08<00:00, 220MB/s]

Some weights of BertForTokenClassification were not initialized from
 You should probably TRAIN this model on a down-stream task to be abl

```
# =====
# 📌 STEP 5: Training
# =====
for epoch in range(EPOCHS):
    print(f"\n🚀 Training Epoch {epoch+1}/{EPOCHS}")
    model.train()
    total_loss = 0
    for batch in train_loader:
        outputs = model(**batch)
        loss = outputs.loss
        total_loss += loss.item()
        loss.backward()
        optimizer.step()
        scheduler.step()
        optimizer.zero_grad()
    print(f"✅ Avg Loss: {total_loss/len(train_loader):.4f}")
```

🚀 Training Epoch 1/3
✅ Avg Loss: 0.7512

🚀 Training Epoch 2/3
✅ Avg Loss: 0.3912

🚀 Training Epoch 3/3
✅ Avg Loss: 0.3230

```
# =====
# 📌 STEP 6: Prediction + Evaluation (mBERT)
# =====

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
model.eval()

all_labels = []
all_preds = []

with torch.no_grad():
    for batch in test_loader:
        input_ids = batch['input_ids'].to(device)
        attention_mask = batch['attention_mask'].to(device)
        labels = batch['labels'].to(device)

        outputs = model(input_ids=input_ids, attention_mask=attention_mask)
        logits = outputs.logits
        predictions = torch.argmax(logits, dim=2)
```

```

    for i in range(len(labels)):
        true_seq, pred_seq = [], []
        for j in range(len(labels[i])):
            # ✅ Skip ignored tokens (-100)
            if labels[i][j].item() == -100:
                continue
            if attention_mask[i][j] == 1:
                true_seq.append(id2label[labels[i][j].item()])
                pred_seq.append(id2label[predictions[i][j].item()])
        all_labels.append(true_seq)
        all_preds.append(pred_seq)

print("✅ Predictions collected successfully!")

# Flatten lists for metric calculation
y_true_flat, y_pred_flat = [], []
for true_seq, pred_seq in zip(all_labels, all_preds):
    for t, p in zip(true_seq, pred_seq):
        y_true_flat.append(t)
        y_pred_flat.append(p)

from sklearn.metrics import precision_recall_fscore_support, accuracy_score

precision, recall, f1, _ = precision_recall_fscore_support(
    y_true_flat, y_pred_flat, average='macro'
)
accuracy = accuracy_score(y_true_flat, y_pred_flat)

print("\n🇺🇸 POS Tagging Evaluation Metrics using mBERT:")
print(f"✅ Accuracy : {accuracy:.4f}")
print(f"✅ Precision : {precision:.4f}")
print(f"✅ Recall : {recall:.4f}")
print(f"✅ F1-score : {f1:.4f}")

```

✅ Predictions collected successfully!

🇺🇸 POS Tagging Evaluation Metrics using mBERT:

✅ Accuracy : 0.8638

✅ Precision : 0.3941

✅ Recall : 0.3980

✅ F1-score : 0.3920

/usr/local/lib/python3.12/dist-packages/sklearn/metrics/_classification.py:1562: UserWarning: The length of the result is less than the length of the input. This may be due to the fact that the metric is not defined for all classes. The metric is not defined for the following classes: [0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99].

```
# =====
# 📌 STEP 7: Save Model
# =====
output_dir = "/content/drive/MyDrive/mbert_pos_model"
model.save_pretrained(output_dir)
tokenizer.save_pretrained(output_dir)
print(f"\n✅ Model saved to {output_dir}")
```

✅ Model saved to /content/drive/MyDrive/mbert_pos_model

```
# =====
# 📌 STEP 8: Gradio Interface for Hindi POS Tagger (mBERT)
# =====
!pip install gradio -q
import gradio as gr
from transformers import AutoTokenizer, AutoModelForTokenClassification
import torch

# ✅ Load trained model from Drive
MODEL_PATH = "/content/drive/MyDrive/mbert_pos_model"
tokenizer = AutoTokenizer.from_pretrained(MODEL_PATH)
model = AutoModelForTokenClassification.from_pretrained(MODEL_PATH)
model.eval()

# ✅ Get label mappings
id2label = model.config.id2label

# ✅ Define prediction function
def mbert_pos_tagger(sentence):
    # Tokenize input
    inputs = tokenizer(sentence.split(), is_split_into_words=True,
                        return_tensors="pt", truncation=True, padding=True)

    # Run model
    with torch.no_grad():
        outputs = model(**inputs)
    predictions = torch.argmax(outputs.logits, dim=2)[0].tolist()

    # Decode tags (skip special tokens)
    tokens = tokenizer.convert_ids_to_tokens(inputs["input_ids"][0])
    results = []
    for token, pred in zip(tokens, predictions):
        if token not in ["[CLS]", "[SEP]", "[PAD]"]:
            results.append(f"{token}/{id2label[pred]}")

    return " ".join(results)
```

```
#  Create Gradio interface
iface = gr.Interface(
    fn=mbert_pos_tagger,
    inputs=gr.Textbox(lines=3, placeholder="हिंदी वाक्य लिखें..."),
    outputs="text",
    title="Hindi POS Tagger (mBERT)",
    description="Enter a Hindi sentence to see Part-of-Speech tags
)

iface.launch(share=True)
```

Colab notebook detected. To show errors in colab notebook, set debug
* Running on public URL: <https://0315012e578f4b95eb.gradio.live>

This share link expires in 1 week. For free permanent hosting and GP

Hindi POS Tagger (mBERT)

Enter a Hindi sentence to see Part-of-Speech tags predicted using Multilingual BERT.

sentence

हिंदी वाक्य लिखें...

Clear

Submit

output

Flag

Built with Gradio  · Settings 

Next steps:  [Deploy to Cloud Run](#)

Start coding or generate with AI.